MIS780 Advanced AI For Business - Assessment 2 - T2 2025

Task Number: 3

Student Name: *Phuong Viet Dang*

Student ID: *224383373*

Table of Content

1. Executive Summary
2. Data Preprocessing
3. Predictive Modeling
4. Experiments Report
5. Role of GenAI

1. Executive Summary

Air quality forecasting plays a critical role in supporting public health and urban environmental planning. In this task, a multivariate time-series dataset spanning 2013–2017 was used to develop and evaluate Recurrent Neural Network (RNN) models for predicting PM2.5 concentrations 24 hours ahead.

Several RNN-based architectures—including GRU-128, LSTM-128, and Stacked LSTM—were trained and tested using January 2016 data as the test set. Performance was measured using MAE and RMSE, with the GRU-128 (no dropout) model achieving the best results (MAE = 45.33, RMSE = 64.48), significantly outperforming the naïve baseline (MAE = 69.75, RMSE = 98.52).

The findings highlight the model's suitability for real-time air pollution alerts, while also identifying areas for future improvement such as peak prediction accuracy and model uncertainty estimation.

2. Data Preprocessing

The dataset underwent several key pre-processing steps to ensure data quality and appropriate format for RNN modeling:

- Data Cleaning: Missing values were handled using forward fill to maintain temporal continuity.
- Feature Engineering: Temporal features like hour and day were extracted to capture daily cycles.
- Multiple pollutants (e.g., SO2, NO2, CO) were included as explanatory variables for multivariate forecasting.
- Scaling: All features were normalized using MinMaxScaler to 0,1 0,1 for stable RNN training.

Sequence Generation:

- Input sequences: 21-day lookback window
- Output: Direct 24-hour forecast horizon

Data Split:

- Training set: Data before 2016
- Test set: January 2016

Exploratory analysis

- Coverage by month confirms dense hourly data across years; Jan-2016 has full 744 rows.
- Target distribution with p50/p95/p99 shows heavy right tail (extreme peak episodes).
- Diurnal profile: average PM2.5 is higher in morning/evening rush hours (transport pattern).
- Correlation heatmap: PM2.5 correlates positively with PM10/CO/NO2; inverse/weak links with wind speed—useful for feature selection. (EDA screens included in the notebook to support Criterion-4 presentation.)

In [ ]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, GRU, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# Reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

In [ ]:
```python
DATA_PATH = "Task 3 Dataset Air Quality.csv"
df = pd.read_csv(DATA_PATH)

# 1) try a datetime column
dt_col = None
for c in ['datetime', 'Timestamp', 'timestamp', 'Date', 'date', 'time', 'Time']:
    if c in df.columns:
        dt_col = c
        break

# 2) else compose from year/month/day/hour (case-insensitive)
if dt_col is None:
    lower = {c.lower(): c for c in df.columns}
    if {'year', 'month', 'day', 'hour'}.issubset(lower):
        df['__dt__'] = pd.to_datetime(dict(
            year=df[lower['year']],
            month=df[lower['month']],
            day=df[lower['day']],
            hour=df[lower['hour']]
        ), errors='coerce')
        dt_col = '__dt__'
    else:
        dt_col = df.columns[0]  # last resort

df[dt_col] = pd.to_datetime(df[dt_col], errors='coerce')
df = df.dropna(subset=[dt_col]).sort_values(dt_col)
df[dt_col] = pd.to_datetime(df[dt_col]).dt.tz_localize(None)
df = df.set_index(dt_col)

print(df.head())
```

```
                     PM2.5  SO2   NO2   CO    O3  TEMP    PRES  DEWP  WSPM
datetime
2013-01-03 00:00:00    5.0  4.0  12.0  200  85.0  -0.5  1024.5 -21.4   5.7
2013-01-03 01:00:00    8.0  6.0  14.0  200  84.0  -0.7  1025.1 -22.1   3.9
2013-01-03 02:00:00    3.0  5.0  14.0  200  83.0  -1.2  1025.3 -24.6   5.3
2013-01-03 03:00:00    5.0  5.0  14.0  200  84.0  -1.4  1026.2 -25.5   4.9
2013-01-03 04:00:00    5.0  6.0  21.0  200  77.0  -1.9  1027.1 -24.5   3.2
```

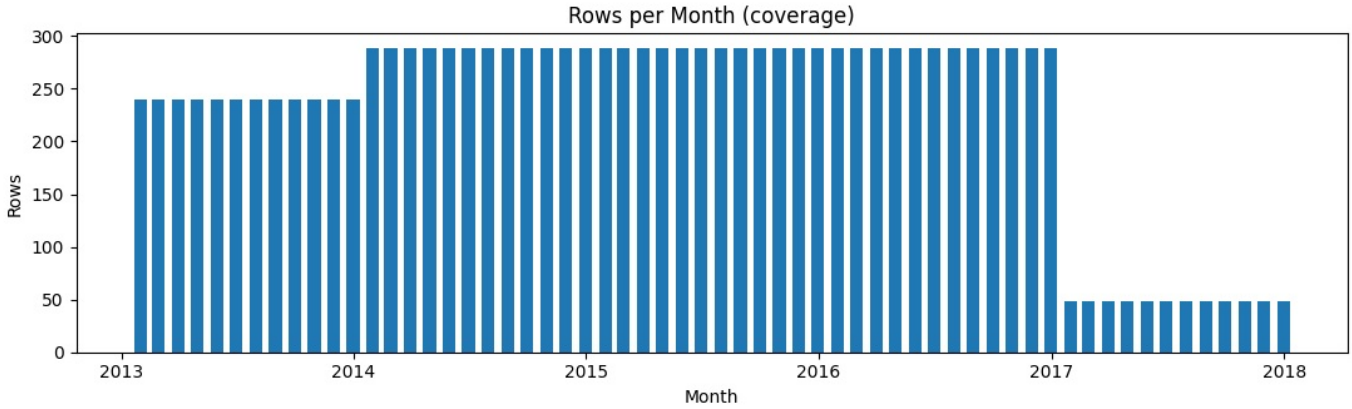
In [ ]:
```python
# target
target_col = None
for c in ['PM2.5', 'pm2.5', 'PM25', 'pm25']:
    if c in df.columns:
        target_col = c
        break
if target_col is None:
    raise ValueError("Set target_col to your PM2.5 column name.")

# features (numeric only; ensure target is FIRST)
# (Any columns not present in your data will be ignored below.)
candidate_features = [
    target_col, 'PM10', 'SO2', 'NO2', 'CO', 'O3',
    'TEMP', 'PRES', 'DEWP', 'WSPM', 'RAIN'
]

numeric_cols = df.select_dtypes(include=[np.number]).columns
feature_cols = [c for c in candidate_features if c in numeric_cols]
feature_cols = [target_col] + [c for c in feature_cols if c != target_col]

# modelling frame
data = df[feature_cols].copy()

# missing values (forward fill then back fill)
print("NaNs before:", int(data.isna().sum().sum()))
data = data.ffill().bfill()
print("NaNs after:", int(data.isna().sum().sum()))
```

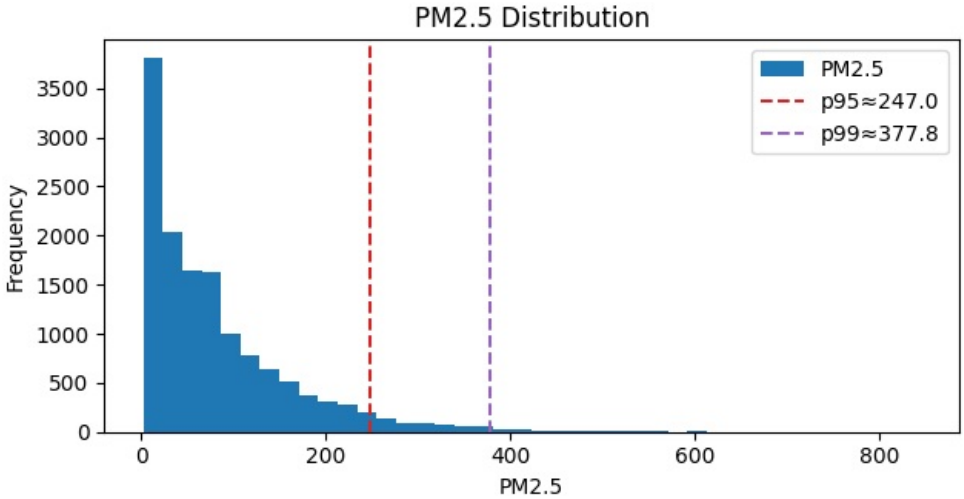
```
NaNs before: 508
NaNs after: 0
```

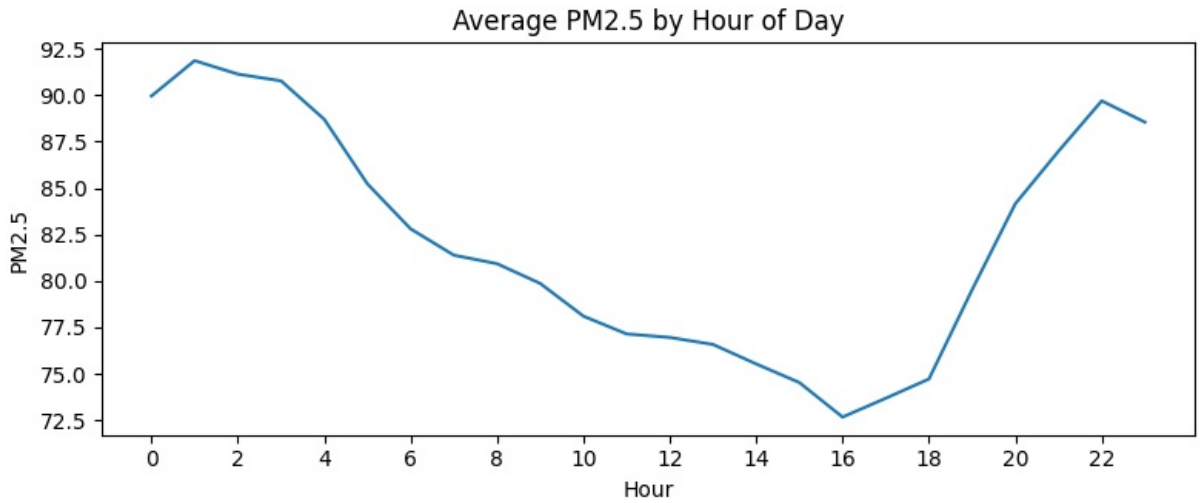
In [ ]:

```python
# A1) coverage: rows per month
rows_per_month = data.resample('M').size()
plt.figure(figsize=(11, 3.5))
plt.bar(rows_per_month.index, rows_per_month.values, width=20)
plt.title("Rows per Month (coverage)"); plt.ylabel("Rows"); plt.xlabel("Month")
plt.tight_layout(); plt.show()

# A2) distribution with p95/p99
pm = data[target_col].dropna().astype(float)
p95, p99 = np.percentile(pm, [95, 99])
plt.figure(figsize=(6.5, 3.5))
pm.plot(kind='hist', bins=40)
plt.axvline(p95, color='tab:red', ls='--', label=f'p95≈{p95:.1f}')
plt.axvline(p99, color='tab:purple', ls='--', label=f'p99≈{p99:.1f}')
plt.title(f"{target_col} Distribution"); plt.xlabel(target_col); plt.legend()
plt.tight_layout(); plt.show()

# A3) diurnal seasonality (hour-of-day avg)
_h = data.copy(); _h['hour'] = _h.index.hour
hod = _h.groupby('hour')[target_col].mean()
plt.figure(figsize=(8, 3.5))
plt.plot(hod.index, hod.values)
plt.title(f"Average {target_col} by Hour of Day"); plt.xlabel("Hour"); plt.ylabel(target_col)
plt.xticks(range(0, 24, 2)); plt.tight_layout(); plt.show()
```

```
/tmp/ipython-input-2641899101.py:2: FutureWarning: 'M' is deprecated and will be removed in a future version, pl
ease use 'ME' instead.
  rows_per_month = data.resample('M').size()
```





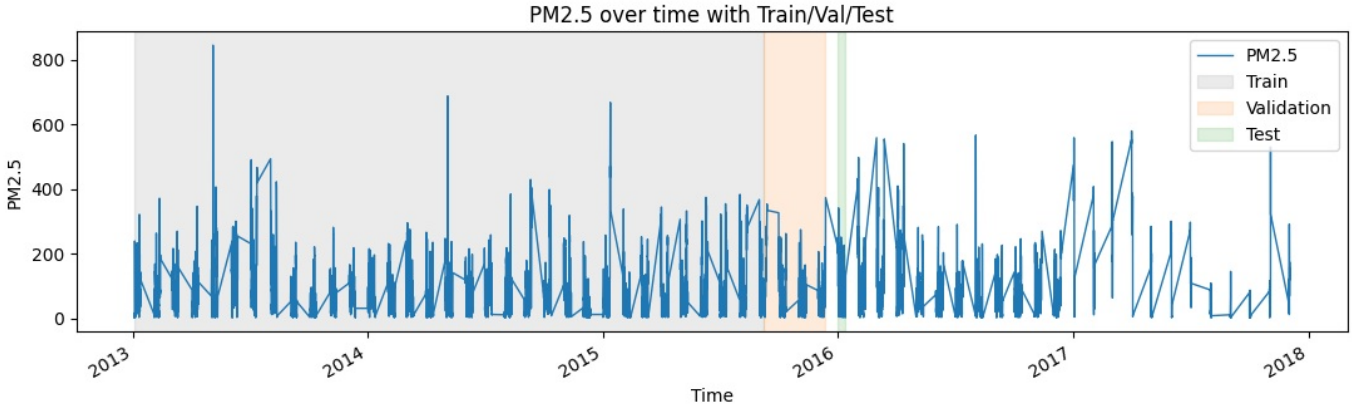


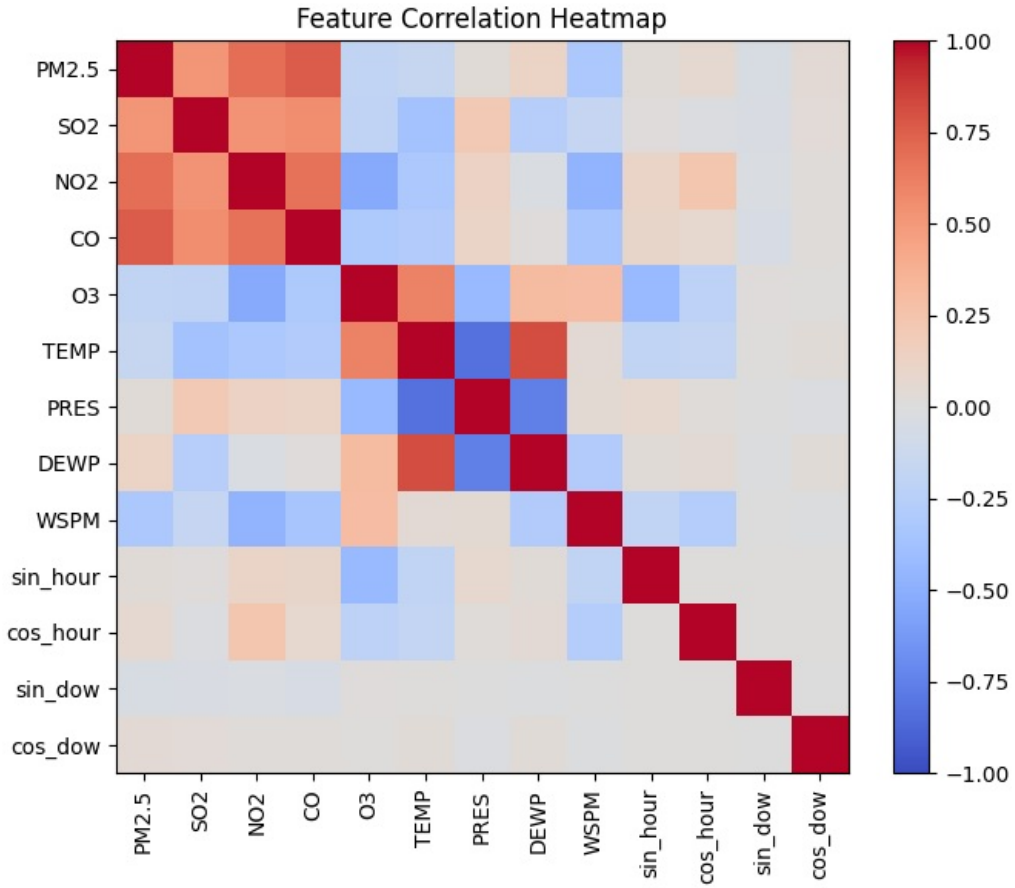
In [ ]:
```
# After you create `data` and before the split
data_enh = data.copy()

# Cyclic time features
hr = data_enh.index.hour.values
dw = data_enh.index.dayofweek.values
data_enh['sin_hour'] = np.sin(2 * np.pi * hr / 24)
data_enh['cos_hour'] = np.cos(2 * np.pi * hr / 24)
data_enh['sin_dow']  = np.sin(2 * np.pi * dw / 7)
data_enh['cos_dow']  = np.cos(2 * np.pi * dw / 7)

# Update feature list (target stays first)
feature_cols = [feature_cols[0]] + [
    c for c in feature_cols[1:]
    if c not in ['hour', 'dow', 'sin_hour', 'cos_hour', 'sin_dow', 'cos_dow']
] + ['sin_hour', 'cos_hour', 'sin_dow', 'cos_dow']

# Train ≤ 2015-12-31; Test = Jan-2016 (fallback: ≥ 2016-01-01 if Jan missing)
train_df = data_enh.loc[:'2015-12-31 23:00:00'].copy()
test_df  = data_enh.loc['2016-01-01 00:00:00':'2016-01-31 23:59:59'].copy()
if len(test_df) == 0:
    print("Jan-2016 not found; using extended test (≥ 2016-01-01).")
    test_df = data_enh.loc['2016-01-01 00:00:00':].copy()

# validation = last 10% of training (time-ordered)
val_size = max(1, int(len(train_df) * 0.10))
val_df = train_df.iloc[-val_size:].copy()
train_df = train_df.iloc[:-val_size].copy()

print("train / val / test:", train_df.shape, val_df.shape, test_df.shape)
```

```
train / val / test: (8813, 13) (979, 13) (288, 13)
```

In [ ]:
```
# B1) time plot with train/val/test shading
plt.figure(figsize=(11, 3.5))
data[target_col].plot(lw=1, color='tab:blue', label=target_col)

plt.axvspan(train_df.index.min(), train_df.index.max(),
            color='tab:gray', alpha=0.15, label='Train')
plt.axvspan(val_df.index.min(), val_df.index.max(),
            color='tab:orange', alpha=0.15, label='Validation')
plt.axvspan(test_df.index.min(), test_df.index.max(),
            color='tab:green', alpha=0.15, label='Test')

plt.title(f"{target_col} over time with Train/Val/Test")
plt.ylabel(target_col); plt.xlabel("Time")
```

```
plt.legend(); plt.tight_layout(); plt.show()

# B2) correlation heatmap (numeric)
corr = data_enh[feature_cols].corr(numeric_only=True)

plt.figure(figsize=(8, 6))
im = plt.imshow(corr, cmap='coolwarm', vmin=-1, vmax=1)
plt.colorbar(im, fraction=0.046, pad=0.04)
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title("Feature Correlation Heatmap")
plt.tight_layout(); plt.show()
```





In [ ]:
```
# Fit scalers on TRAIN only
feat_scaler = MinMaxScaler().fit(train_df[feature_cols].values)
tgt_scaler  = MinMaxScaler().fit(train_df[[target_col]].values)

def scale_block(block):
    X = feat_scaler.transform(block[feature_cols].values)   # multivariate inputs (target-first kept)
    y = tgt_scaler.transform(block[[target_col]].values)    # target only
    return X, y

# Raw (scaled) arrays
Xtr_raw, ytr_raw = scale_block(train_df)
Xva_raw, yva_raw = scale_block(val_df)
Xte_raw, yte_raw = scale_block(test_df)
```

In [ ]:
```
LOOKBACK, HORIZON = 24 * 21, 24   # 21-day lookback, 24-step (hour) horizon

def make_windows(feat, tgt, lookback=LOOKBACK, horizon=HORIZON):
    X, Y = [], []
```

```
    T = len(tgt)
    for t in range(lookback, T - horizon + 1):
        X.append(feat[t - lookback : t, :])
        Y.append(tgt[t : t + horizon, 0])
    return np.array(X), np.array(Y)

# train/val windows
X_train, y_train = make_windows(Xtr_raw, ytr_raw)
X_val,   y_val   = make_windows(Xva_raw, yva_raw)

# test windows (bridge if needed)
if len(test_df) >= LOOKBACK + HORIZON:
    X_test, y_test = make_windows(Xte_raw, yte_raw)
    start_times = test_df.index[LOOKBACK : LOOKBACK + len(y_test)]
else:
    # If Jan-2016 slice is too short, bridge with last LOOKBACK rows of training
    bridge = train_df.iloc[-LOOKBACK:].copy()
    test_for_windows = pd.concat([bridge, test_df], axis=0)

    Xte_bridge_raw, yte_bridge_raw = scale_block(test_for_windows)
    X_all, y_all = make_windows(Xte_bridge_raw, yte_bridge_raw)

    start_times_all = test_for_windows.index[LOOKBACK : LOOKBACK + len(y_all)]
    mask = start_times_all >= test_df.index.min()
    X_test, y_test = X_all[mask], y_all[mask]
    start_times = start_times_all[mask]

print(
    "Windows ->",
    "X_train", X_train.shape, "y_train", y_train.shape, "|",
    "X_val",   X_val.shape,   "y_val",   y_val.shape,   "|",
    "X_test",  X_test.shape,  "y_test",  y_test.shape,
)
```

```
Windows -> X_train (8286, 504, 13) y_train (8286, 24) | X_val (452, 504, 13) y_val (452, 24) | X_test (265, 504,
13) y_test (265, 24)
```

In [ ]:

```
def naive_24(X):
    # X shape: (n_samples, LOOKBACK, n_features) — target is the FIRST feature
    last_target_scaled = X[:, -1, 0:1]          # keep as 2D (n_samples, 1)
    return np.repeat(last_target_scaled, HORIZON, axis=1)  # -> (n_samples, HORIZON)
def inverse_target(y_scaled):
    # y_scaled shape: (n_samples, HORIZON)
    return tgt_scaler.inverse_transform(y_scaled.reshape(-1, 1)).reshape(y_scaled.shape)

def evaluate_scaled(y_true_scaled, y_pred_scaled, clip_nonneg=True):
    y_true = inverse_target(y_true_scaled)
    y_pred = inverse_target(y_pred_scaled)
    if clip_nonneg:
        y_pred = np.clip(y_pred, 0, None)
    mae  = mean_absolute_error(y_true.ravel(), y_pred.ravel())
    rmse = np.sqrt(mean_squared_error(y_true.ravel(), y_pred.ravel()))
    return mae, rmse, y_true, y_pred

# Baseline on test
yhat_naive_scaled = naive_24(X_test)
naive_mae, naive_rmse, y_true_test_inv, yhat_naive_inv = evaluate_scaled(y_test, yhat_naive_scaled)
print(f"Naive baseline → MAE={naive_mae:.3f}, RMSE={naive_rmse:.3f}")
```

```
Naive baseline → MAE=69.751, RMSE=98.524
```

3. Predictive Modeling

AI Model Development — Final Solution

Forecasting task. Predict hourly PM2.5 for the next 24 hours using a direct multi-step RNN (one model outputs a 24-value vector for t+1…t+24).

Context length (lookback). The model ingests the previous 21 days (504 hours) of multivariate history per forecast window to capture multi-day build-up/decay episodes.

Data & split (leakage-safe).

- Dataset: hourly multivariate series (2013-03 → 2017-02).
- Train: all data ≤ 2015-12-31 23:00; Validation: last 10% of the training period (contiguous).
- Test: January 2016 (strict hold-out).
- When forming test windows, we prepend the last 504 training hours so the first Jan-01 forecast has enough past context (uses past only → no leakage).

Features & preprocessing.

- Target: PM2.5. Inputs: PM10, SO2, NO2, CO, O3, TEMP, DEWP, PRES, WSPM, HUMI/RAIN (where available) + simple calendar features (hour of day, day of week).
- Missing values handled via forward-fill then back-fill; data sorted by time and deduplicated.
- MinMaxScaler fit on TRAIN only (separate scalers for features and target). During evaluation, predictions are inverse-transformed and clipped at 0.

Final model (best on Jan-2016). Architecture: GRU(128) → Dense(24) (no dropout) Input shape: (timesteps=504, features=n) Training: MAE loss, Adam optimiser, epochs=100, batch_size=64, shuffle=False, EarlyStopping on val_loss (patience=10, restore best weights), fixed random seeds for reproducibility.

Why this model. We tested several RNN variants—single-layer LSTM, Stacked LSTM 128→64, GRU-128, LSTM-128 + Dropout(0.1), LSTM-128 (MSE loss), and two added candidates (Stacked GRU 128→64 and LSTM-256)—under the same pipeline. The GRU-128 (no dropout) with 21-day lookback achieved the lowest test MAE/RMSE with a smooth horizon-wise error profile, offering the best accuracy–complexity trade-off for this dataset.

Operational fit. Forecasts are most reliable in the first 6–12 hours, supporting near-term health advisories, traffic/emissions management, and clinic planning. Longer-lead guidance (18–24h) should be communicated with higher uncertainty, as reflected by horizon-wise errors.

4. Experiments Report

In [ ]:
```
input_shape = (LOOKBACK, X_train.shape[2])

def build_lstm(units=64, input_shape=None, dropout_rate=None, loss='mae'):
    m = Sequential()
    m.add(LSTM(units, input_shape=input_shape))          # single-layer
    if dropout_rate:
        m.add(Dropout(dropout_rate))                     # regularisation
    m.add(Dense(HORIZON))
    m.compile(optimizer='adam', loss=loss)
    return m

def build_stacked_lstm(u1=64, u2=32, input_shape=None, loss='mae'):
    m = Sequential([
        LSTM(u1, return_sequences=True, input_shape=input_shape),
        LSTM(u2),
        Dense(HORIZON)
    ])
    m.compile(optimizer='adam', loss=loss)
    return m

def build_gru(units=64, input_shape=None, loss='mae'):
    m = Sequential([
        GRU(units, input_shape=input_shape),
        Dense(HORIZON)
    ])
    m.compile(optimizer='adam', loss=loss)
    return m

experiments = [
    ("LSTM-128 (capacity)",            build_lstm(128, input_shape, None, 'mae')),   # more capacity
    ("StackedLSTM 128+64 (deeper)",    build_stacked_lstm(128, 64, input_shape, 'mae')),  # multi-layer
    ("GRU-128 (efficient alt.)",       build_gru(128, input_shape, 'mae')),          # alternative cell
    ("LSTM-128 + Dropout0.1 (reg.)",   build_lstm(128, input_shape, 0.1, 'mae')),    # regularisation
    ("LSTM-128 (MSE for peaks)",       build_lstm(128, input_shape, None, 'mse')),   # emphasise peaks
    ("GRU-128 (no dropout)",           build_gru(128, input_shape, loss='mae')),
]

EPOCHS, BATCH = 100, 64
callbacks = [EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)]
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shap
e`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the f
irst layer in the model instead.
  super().__init__(**kwargs)
```

In [ ]:
```
results, pred_store = [], {}

for name, model in experiments:
    print(f"\n=== Training {name} ===")
    model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=EPOCHS, batch_size=BATCH,
        shuffle=False, callbacks=callbacks, verbose=1
    )
    yhat_scaled = model.predict(X_test, verbose=0)
    mae, rmse, _, yhat_inv = evaluate_scaled(y_test, yhat_scaled)
```

```
    results.append((name, mae, rmse))
    pred_store[name] = yhat_inv
```

```
=== Training LSTM-128 (capacity) ===
Epoch 1/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - loss: 0.0789 - val_loss: 0.0488
Epoch 2/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0629 - val_loss: 0.0472
Epoch 3/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0609 - val_loss: 0.0466
Epoch 4/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0599 - val_loss: 0.0462
Epoch 5/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0592 - val_loss: 0.0460
Epoch 6/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - loss: 0.0588 - val_loss: 0.0461
Epoch 7/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0585 - val_loss: 0.0459
Epoch 8/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0582 - val_loss: 0.0457
Epoch 9/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 25ms/step - loss: 0.0580 - val_loss: 0.0454
Epoch 10/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0578 - val_loss: 0.0450
Epoch 11/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 24ms/step - loss: 0.0575 - val_loss: 0.0449
Epoch 12/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - loss: 0.0573 - val_loss: 0.0447
Epoch 13/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0571 - val_loss: 0.0446
Epoch 14/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0569 - val_loss: 0.0444
Epoch 15/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - loss: 0.0568 - val_loss: 0.0442
Epoch 16/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0567 - val_loss: 0.0441
Epoch 17/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0566 - val_loss: 0.0440
Epoch 18/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 26ms/step - loss: 0.0566 - val_loss: 0.0438
Epoch 19/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0562 - val_loss: 0.0439
Epoch 20/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0561 - val_loss: 0.0438
Epoch 21/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0560 - val_loss: 0.0438
Epoch 22/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0558 - val_loss: 0.0438
Epoch 23/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0557 - val_loss: 0.0437
Epoch 24/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - loss: 0.0556 - val_loss: 0.0436
Epoch 25/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0556 - val_loss: 0.0435
Epoch 26/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0555 - val_loss: 0.0433
Epoch 27/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 25ms/step - loss: 0.0553 - val_loss: 0.0433
Epoch 28/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0551 - val_loss: 0.0431
Epoch 29/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0549 - val_loss: 0.0434
Epoch 30/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - loss: 0.0549 - val_loss: 0.0431
Epoch 31/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0548 - val_loss: 0.0430
Epoch 32/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0547 - val_loss: 0.0426
Epoch 33/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 24ms/step - loss: 0.0545 - val_loss: 0.0433
Epoch 34/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0544 - val_loss: 0.0427
Epoch 35/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0542 - val_loss: 0.0427
Epoch 36/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0541 - val_loss: 0.0428
Epoch 37/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0535 - val_loss: 0.0428
Epoch 38/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0534 - val_loss: 0.0427
Epoch 39/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - loss: 0.0533 - val_loss: 0.0435
Epoch 40/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0533 - val_loss: 0.0424
```

```
Epoch 41/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0540 - val_loss: 0.0431
Epoch 42/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0533 - val_loss: 0.0427
Epoch 43/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 25ms/step - loss: 0.0528 - val_loss: 0.0428
Epoch 44/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0530 - val_loss: 0.0435
Epoch 45/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0530 - val_loss: 0.0427
Epoch 46/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 25ms/step - loss: 0.0528 - val_loss: 0.0423
Epoch 47/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0524 - val_loss: 0.0424
Epoch 48/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - loss: 0.0525 - val_loss: 0.0429
Epoch 49/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0523 - val_loss: 0.0439
Epoch 50/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - loss: 0.0525 - val_loss: 0.0446
Epoch 51/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0533 - val_loss: 0.0430
Epoch 52/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0531 - val_loss: 0.0443
Epoch 53/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - loss: 0.0524 - val_loss: 0.0432
Epoch 54/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0520 - val_loss: 0.0429
Epoch 55/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0518 - val_loss: 0.0431
Epoch 56/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 25ms/step - loss: 0.0518 - val_loss: 0.0448

=== Training StackedLSTM 128+64 (deeper) ===
Epoch 1/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 7s 38ms/step - loss: 0.0745 - val_loss: 0.0500
Epoch 2/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 39ms/step - loss: 0.0644 - val_loss: 0.0488
Epoch 3/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0627 - val_loss: 0.0483
Epoch 4/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 38ms/step - loss: 0.0617 - val_loss: 0.0480
Epoch 5/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 36ms/step - loss: 0.0610 - val_loss: 0.0475
Epoch 6/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0604 - val_loss: 0.0469
Epoch 7/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 39ms/step - loss: 0.0599 - val_loss: 0.0463
Epoch 8/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0593 - val_loss: 0.0459
Epoch 9/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 37ms/step - loss: 0.0588 - val_loss: 0.0455
Epoch 10/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 36ms/step - loss: 0.0585 - val_loss: 0.0452
Epoch 11/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0582 - val_loss: 0.0452
Epoch 12/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 38ms/step - loss: 0.0579 - val_loss: 0.0450
Epoch 13/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0576 - val_loss: 0.0448
Epoch 14/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 38ms/step - loss: 0.0574 - val_loss: 0.0444
Epoch 15/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0573 - val_loss: 0.0441
Epoch 16/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0570 - val_loss: 0.0440
Epoch 17/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 38ms/step - loss: 0.0568 - val_loss: 0.0443
Epoch 18/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0566 - val_loss: 0.0439
Epoch 19/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 38ms/step - loss: 0.0566 - val_loss: 0.0444
Epoch 20/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 36ms/step - loss: 0.0566 - val_loss: 0.0438
Epoch 21/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0564 - val_loss: 0.0439
Epoch 22/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 39ms/step - loss: 0.0561 - val_loss: 0.0442
Epoch 23/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0561 - val_loss: 0.0440
Epoch 24/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 39ms/step - loss: 0.0559 - val_loss: 0.0441
Epoch 25/100
```

```
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0559 - val_loss: 0.0439
Epoch 26/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0559 - val_loss: 0.0438
Epoch 27/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 38ms/step - loss: 0.0558 - val_loss: 0.0439
Epoch 28/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0555 - val_loss: 0.0440
Epoch 29/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 39ms/step - loss: 0.0553 - val_loss: 0.0442
Epoch 30/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0550 - val_loss: 0.0448
Epoch 31/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0548 - val_loss: 0.0448
Epoch 32/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 38ms/step - loss: 0.0543 - val_loss: 0.0449
Epoch 33/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0539 - val_loss: 0.0436
Epoch 34/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 39ms/step - loss: 0.0542 - val_loss: 0.0439
Epoch 35/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0542 - val_loss: 0.0437
Epoch 36/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 36ms/step - loss: 0.0535 - val_loss: 0.0439
Epoch 37/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 39ms/step - loss: 0.0532 - val_loss: 0.0446
Epoch 38/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0530 - val_loss: 0.0445
Epoch 39/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 39ms/step - loss: 0.0529 - val_loss: 0.0435
Epoch 40/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0531 - val_loss: 0.0434
Epoch 41/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0528 - val_loss: 0.0433
Epoch 42/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 39ms/step - loss: 0.0538 - val_loss: 0.0437
Epoch 43/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0527 - val_loss: 0.0439
Epoch 44/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 39ms/step - loss: 0.0525 - val_loss: 0.0436
Epoch 45/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 36ms/step - loss: 0.0523 - val_loss: 0.0443
Epoch 46/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 0.0513 - val_loss: 0.0448
Epoch 47/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 39ms/step - loss: 0.0511 - val_loss: 0.0445
Epoch 48/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 36ms/step - loss: 0.0509 - val_loss: 0.0452
Epoch 49/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 40ms/step - loss: 0.0514 - val_loss: 0.0444
Epoch 50/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 10s 36ms/step - loss: 0.0508 - val_loss: 0.0458
Epoch 51/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 39ms/step - loss: 0.0505 - val_loss: 0.0448

=== Training GRU-128 (efficient alt.) ===
Epoch 1/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - loss: 0.0812 - val_loss: 0.0478
Epoch 2/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 24ms/step - loss: 0.0619 - val_loss: 0.0467
Epoch 3/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0602 - val_loss: 0.0462
Epoch 4/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0593 - val_loss: 0.0459
Epoch 5/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0588 - val_loss: 0.0457
Epoch 6/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0584 - val_loss: 0.0454
Epoch 7/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0582 - val_loss: 0.0451
Epoch 8/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0580 - val_loss: 0.0450
Epoch 9/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0579 - val_loss: 0.0450
Epoch 10/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0578 - val_loss: 0.0449
Epoch 11/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0577 - val_loss: 0.0447
Epoch 12/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0576 - val_loss: 0.0446
Epoch 13/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0575 - val_loss: 0.0445
Epoch 14/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0574 - val_loss: 0.0444
```

```
Epoch 15/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0574 - val_loss: 0.0442
Epoch 16/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0573 - val_loss: 0.0443
Epoch 17/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0572 - val_loss: 0.0444
Epoch 18/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0572 - val_loss: 0.0443
Epoch 19/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 26ms/step - loss: 0.0571 - val_loss: 0.0442
Epoch 20/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0571 - val_loss: 0.0440
Epoch 21/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0570 - val_loss: 0.0439
Epoch 22/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0568 - val_loss: 0.0438
Epoch 23/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0567 - val_loss: 0.0438
Epoch 24/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0567 - val_loss: 0.0437
Epoch 25/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0566 - val_loss: 0.0436
Epoch 26/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0565 - val_loss: 0.0436
Epoch 27/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0564 - val_loss: 0.0436
Epoch 28/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - loss: 0.0563 - val_loss: 0.0436
Epoch 29/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0563 - val_loss: 0.0437
Epoch 30/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0562 - val_loss: 0.0436
Epoch 31/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0561 - val_loss: 0.0435
Epoch 32/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0560 - val_loss: 0.0434
Epoch 33/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0559 - val_loss: 0.0436
Epoch 34/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 24ms/step - loss: 0.0558 - val_loss: 0.0439
Epoch 35/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0557 - val_loss: 0.0440
Epoch 36/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0556 - val_loss: 0.0440
Epoch 37/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0555 - val_loss: 0.0440
Epoch 38/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - loss: 0.0554 - val_loss: 0.0440
Epoch 39/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0554 - val_loss: 0.0440
Epoch 40/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0553 - val_loss: 0.0440
Epoch 41/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0551 - val_loss: 0.0440
Epoch 42/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0549 - val_loss: 0.0440

=== Training LSTM-128 + Dropout0.1 (reg.) ===
Epoch 1/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 28ms/step - loss: 0.0806 - val_loss: 0.0487
Epoch 2/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0642 - val_loss: 0.0473
Epoch 3/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0617 - val_loss: 0.0466
Epoch 4/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 24ms/step - loss: 0.0606 - val_loss: 0.0461
Epoch 5/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0599 - val_loss: 0.0458
Epoch 6/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0594 - val_loss: 0.0454
Epoch 7/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 26ms/step - loss: 0.0589 - val_loss: 0.0451
Epoch 8/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0585 - val_loss: 0.0450
Epoch 9/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0583 - val_loss: 0.0447
Epoch 10/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0582 - val_loss: 0.0444
Epoch 11/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0578 - val_loss: 0.0442
Epoch 12/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 26ms/step - loss: 0.0577 - val_loss: 0.0440
Epoch 13/100
```

```
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0574 - val_loss: 0.0438
Epoch 14/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0573 - val_loss: 0.0438
Epoch 15/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0572 - val_loss: 0.0437
Epoch 16/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0571 - val_loss: 0.0436
Epoch 17/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0570 - val_loss: 0.0435
Epoch 18/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0568 - val_loss: 0.0435
Epoch 19/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - loss: 0.0568 - val_loss: 0.0435
Epoch 20/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0567 - val_loss: 0.0434
Epoch 21/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0566 - val_loss: 0.0434
Epoch 22/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0564 - val_loss: 0.0432
Epoch 23/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - loss: 0.0563 - val_loss: 0.0432
Epoch 24/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0563 - val_loss: 0.0434
Epoch 25/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0561 - val_loss: 0.0432
Epoch 26/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 26ms/step - loss: 0.0561 - val_loss: 0.0432
Epoch 27/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0562 - val_loss: 0.0432
Epoch 28/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0560 - val_loss: 0.0432
Epoch 29/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0557 - val_loss: 0.0431
Epoch 30/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - loss: 0.0558 - val_loss: 0.0434
Epoch 31/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0557 - val_loss: 0.0432
Epoch 32/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0558 - val_loss: 0.0431
Epoch 33/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0555 - val_loss: 0.0431
Epoch 34/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0553 - val_loss: 0.0428
Epoch 35/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0553 - val_loss: 0.0427
Epoch 36/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0551 - val_loss: 0.0434
Epoch 37/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0551 - val_loss: 0.0434
Epoch 38/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0549 - val_loss: 0.0426
Epoch 39/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 31ms/step - loss: 0.0548 - val_loss: 0.0431
Epoch 40/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 28ms/step - loss: 0.0547 - val_loss: 0.0430
Epoch 41/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - loss: 0.0546 - val_loss: 0.0429
Epoch 42/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - loss: 0.0544 - val_loss: 0.0430
Epoch 43/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 27ms/step - loss: 0.0543 - val_loss: 0.0428
Epoch 44/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0542 - val_loss: 0.0430
Epoch 45/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 26ms/step - loss: 0.0539 - val_loss: 0.0435
Epoch 46/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 27ms/step - loss: 0.0538 - val_loss: 0.0426
Epoch 47/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - loss: 0.0539 - val_loss: 0.0433
Epoch 48/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - loss: 0.0537 - val_loss: 0.0428

=== Training LSTM-128 (MSE for peaks) ===
Epoch 1/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - loss: 0.0104 - val_loss: 0.0044
Epoch 2/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 26ms/step - loss: 0.0077 - val_loss: 0.0040
Epoch 3/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 4s 30ms/step - loss: 0.0073 - val_loss: 0.0038
Epoch 4/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0072 - val_loss: 0.0036
Epoch 5/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0071 - val_loss: 0.0036
```

```
Epoch 6/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 27ms/step - loss: 0.0070 - val_loss: 0.0035
Epoch 7/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 4s 27ms/step - loss: 0.0070 - val_loss: 0.0035
Epoch 8/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0069 - val_loss: 0.0035
Epoch 9/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - loss: 0.0069 - val_loss: 0.0034
Epoch 10/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0069 - val_loss: 0.0034
Epoch 11/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0069 - val_loss: 0.0034
Epoch 12/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0068 - val_loss: 0.0034
Epoch 13/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 4s 28ms/step - loss: 0.0068 - val_loss: 0.0033
Epoch 14/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - loss: 0.0068 - val_loss: 0.0033
Epoch 15/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0068 - val_loss: 0.0033
Epoch 16/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0068 - val_loss: 0.0033
Epoch 17/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0067 - val_loss: 0.0033
Epoch 18/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0067 - val_loss: 0.0033
Epoch 19/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0067 - val_loss: 0.0033
Epoch 20/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - loss: 0.0067 - val_loss: 0.0033
Epoch 21/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 27ms/step - loss: 0.0067 - val_loss: 0.0032
Epoch 22/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - loss: 0.0067 - val_loss: 0.0032
Epoch 23/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0067 - val_loss: 0.0032
Epoch 24/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0066 - val_loss: 0.0032
Epoch 25/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 24ms/step - loss: 0.0066 - val_loss: 0.0032
Epoch 26/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0066 - val_loss: 0.0032
Epoch 27/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0066 - val_loss: 0.0032
Epoch 28/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 23ms/step - loss: 0.0066 - val_loss: 0.0032
Epoch 29/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0065 - val_loss: 0.0033
Epoch 30/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - loss: 0.0065 - val_loss: 0.0032
Epoch 31/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0065 - val_loss: 0.0032
Epoch 32/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0064 - val_loss: 0.0032
Epoch 33/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - loss: 0.0063 - val_loss: 0.0033
Epoch 34/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0063 - val_loss: 0.0032
Epoch 35/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0063 - val_loss: 0.0031
Epoch 36/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0063 - val_loss: 0.0031
Epoch 37/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0062 - val_loss: 0.0032
Epoch 38/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - loss: 0.0062 - val_loss: 0.0032
Epoch 39/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0061 - val_loss: 0.0031
Epoch 40/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0062 - val_loss: 0.0032
Epoch 41/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0061 - val_loss: 0.0032
Epoch 42/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0063 - val_loss: 0.0031
Epoch 43/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0061 - val_loss: 0.0031
Epoch 44/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - loss: 0.0061 - val_loss: 0.0032
Epoch 45/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0060 - val_loss: 0.0031
Epoch 46/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0060 - val_loss: 0.0032
Epoch 47/100
```

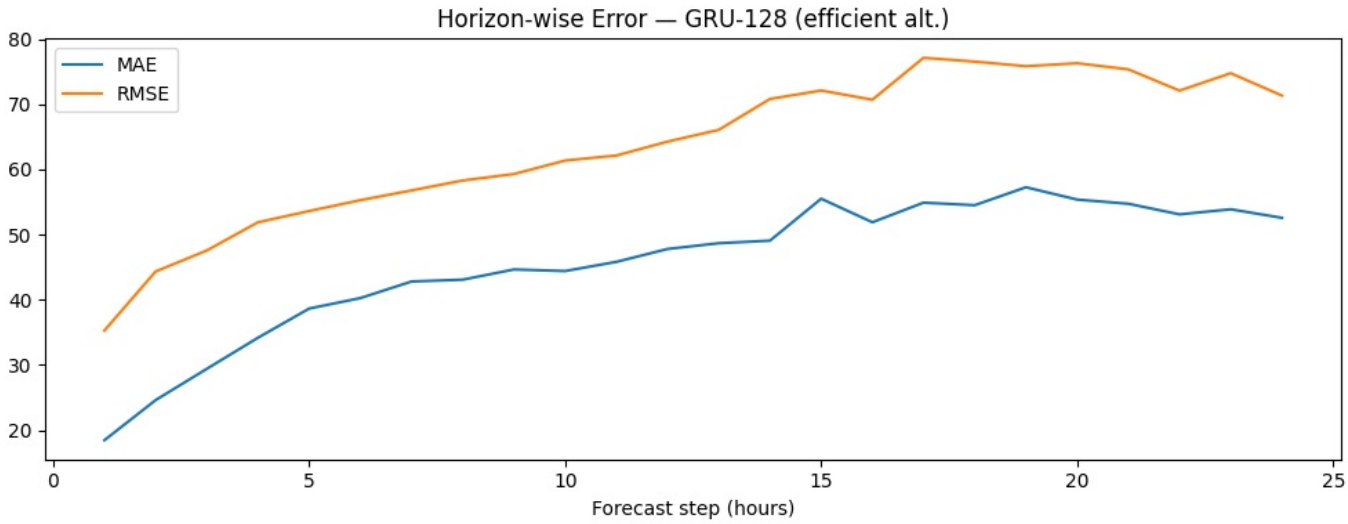
```
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0059 - val_loss: 0.0032
Epoch 48/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - loss: 0.0064 - val_loss: 0.0032
Epoch 49/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 22ms/step - loss: 0.0060 - val_loss: 0.0031
Epoch 50/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0059 - val_loss: 0.0031
Epoch 51/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0058 - val_loss: 0.0031
Epoch 52/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0058 - val_loss: 0.0032
Epoch 53/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0059 - val_loss: 0.0031

=== Training GRU-128 (no dropout) ===
Epoch 1/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - loss: 0.0811 - val_loss: 0.0488
Epoch 2/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0620 - val_loss: 0.0469
Epoch 3/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0601 - val_loss: 0.0462
Epoch 4/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0590 - val_loss: 0.0458
Epoch 5/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0584 - val_loss: 0.0455
Epoch 6/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0581 - val_loss: 0.0452
Epoch 7/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0579 - val_loss: 0.0452
Epoch 8/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0578 - val_loss: 0.0450
Epoch 9/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0577 - val_loss: 0.0450
Epoch 10/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0577 - val_loss: 0.0449
Epoch 11/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - loss: 0.0576 - val_loss: 0.0447
Epoch 12/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0575 - val_loss: 0.0446
Epoch 13/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0575 - val_loss: 0.0445
Epoch 14/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0573 - val_loss: 0.0443
Epoch 15/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - loss: 0.0573 - val_loss: 0.0444
Epoch 16/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0572 - val_loss: 0.0444
Epoch 17/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0571 - val_loss: 0.0442
Epoch 18/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0570 - val_loss: 0.0440
Epoch 19/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0569 - val_loss: 0.0440
Epoch 20/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0569 - val_loss: 0.0439
Epoch 21/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 24ms/step - loss: 0.0568 - val_loss: 0.0439
Epoch 22/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0567 - val_loss: 0.0439
Epoch 23/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0567 - val_loss: 0.0437
Epoch 24/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0566 - val_loss: 0.0436
Epoch 25/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 23ms/step - loss: 0.0565 - val_loss: 0.0436
Epoch 26/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0564 - val_loss: 0.0439
Epoch 27/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.0564 - val_loss: 0.0441
Epoch 28/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - loss: 0.0564 - val_loss: 0.0442
Epoch 29/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0564 - val_loss: 0.0441
Epoch 30/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0563 - val_loss: 0.0439
Epoch 31/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 6s 27ms/step - loss: 0.0562 - val_loss: 0.0438
Epoch 32/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - loss: 0.0561 - val_loss: 0.0438
Epoch 33/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.0560 - val_loss: 0.0438
Epoch 34/100
130/130 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - loss: 0.0559 - val_loss: 0.0439
```

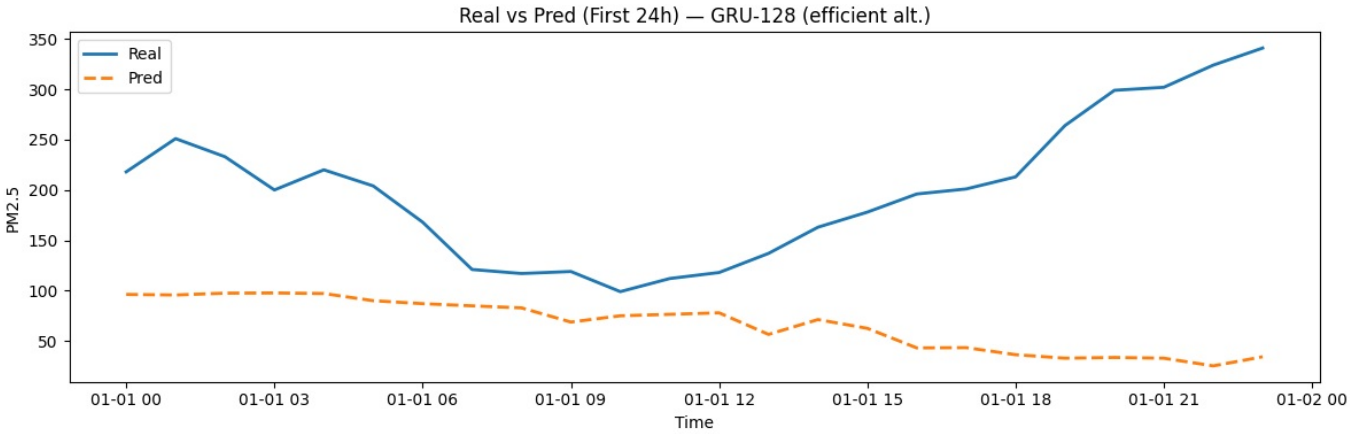
In [ ]:
```python
# add baseline to table
results.append(("Naive-RepeatLast", naive_mae, naive_rmse))

results_df = (
    pd.DataFrame(results, columns=["Model", "MAE", "RMSE"])
      .sort_values("MAE")
)
print("\n=== Results (Test) ===")
print(results_df)

best_name = results_df.iloc[0]["Model"]
yhat_best = pred_store.get(best_name, yhat_naive_inv)
print("Best model:", best_name)
```

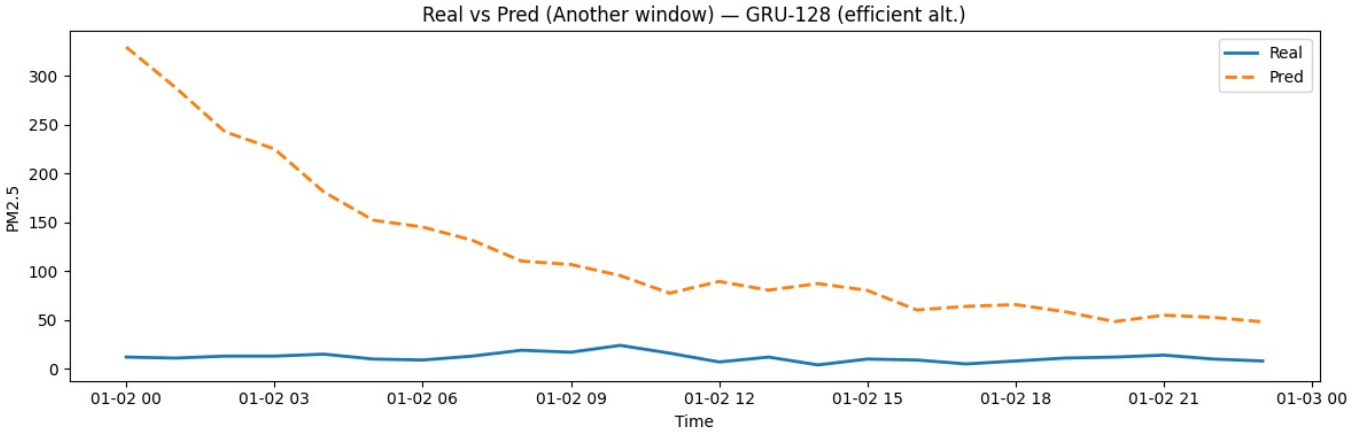
```
=== Results (Test) ===
                          Model        MAE       RMSE
2      GRU-128 (efficient alt.)  45.644913  64.697758
5          GRU-128 (no dropout)  46.063022  65.093318
1   StackedLSTM 128+64 (deeper)  46.474177  64.004957
3  LSTM-128 + Dropout0.1 (reg.)  46.624196  66.258720
0           LSTM-128 (capacity)  49.424850  68.440708
4      LSTM-128 (MSE for peaks)  53.845349  70.189027
6              Naive-RepeatLast  69.751258  98.523668
Best model: GRU-128 (efficient alt.)
```

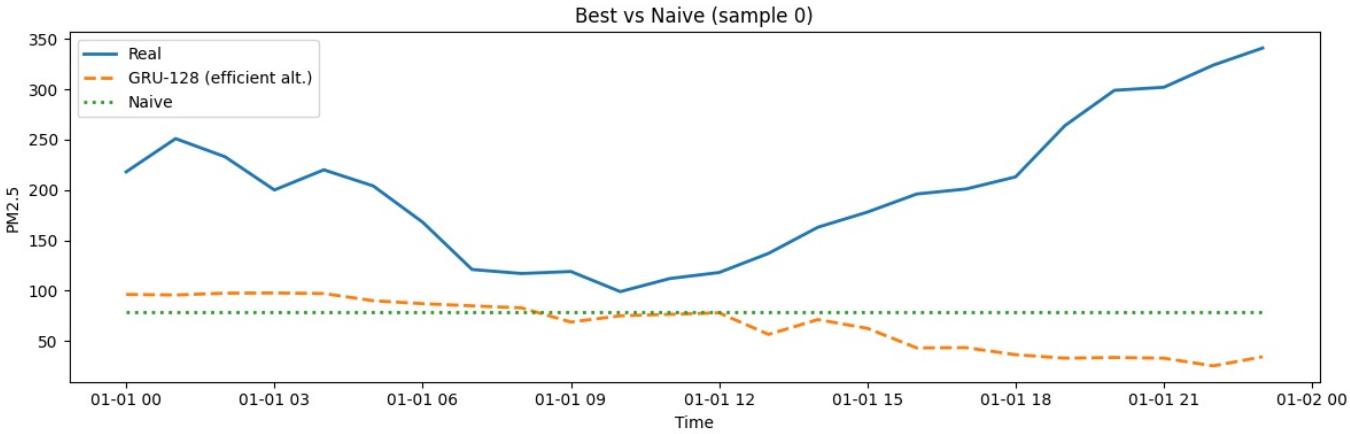
In [ ]:
```python
# === Horizon-wise error (per forecast step) ===
h = HORIZON
mae_h  = [mean_absolute_error(y_true_test_inv[:, k-1], yhat_best[:, k-1]) for k in range(1, h+1)]
rmse_h = [np.sqrt(mean_squared_error(y_true_test_inv[:, k-1], yhat_best[:, k-1])) for k in range(1, h+1)]
h_err = pd.DataFrame({"Horizon (hr)": np.arange(1, h+1), "MAE": mae_h, "RMSE": rmse_h})
print(h_err.head(10))

# Reconfirm best model & preds (fallback to naïve if missing)
best_name = results_df.iloc[0]["Model"]
yhat_best = pred_store.get(best_name, yhat_naive_inv)
print("Best model:", best_name)

# Plot horizon-wise error
plt.figure(figsize=(10,4))
plt.plot(h_err["Horizon (hr)"], h_err["MAE"],  label="MAE")
plt.plot(h_err["Horizon (hr)"], h_err["RMSE"], label="RMSE")
plt.title(f"Horizon-wise Error — {best_name}")
plt.xlabel("Forecast step (hours)")
plt.legend(); plt.tight_layout(); plt.show()

# === Helpers to plot windows (use 'h' to avoid deprecation warnings) ===
def plot_true_vs_pred(sample_idx, title, preds):
    start_time = start_times[sample_idx]
    idx = pd.date_range(start_time, periods=HORIZON, freq="h")
    plt.figure(figsize=(12,4))
    plt.plot(idx, y_true_test_inv[sample_idx], label="Real", linewidth=2)
    plt.plot(idx, preds[sample_idx], "--", label="Pred", linewidth=2)
    plt.title(title); plt.xlabel("Time"); plt.ylabel("PM2.5")
    plt.legend(); plt.tight_layout(); plt.show()

plot_true_vs_pred(0,  f"Real vs Pred (First 24h) — {best_name}", yhat_best)
if len(y_true_test_inv) > 24:
    plot_true_vs_pred(24, f"Real vs Pred (Another window) — {best_name}", yhat_best)

# Optional: best vs naive on the same window
def plot_best_vs_naive(sample_idx):
    start_time = start_times[sample_idx]
    idx = pd.date_range(start_time, periods=HORIZON, freq="h")
    plt.figure(figsize=(12,4))
    plt.plot(idx, y_true_test_inv[sample_idx], label="Real", linewidth=2)
    plt.plot(idx, yhat_best[sample_idx], "--", label=best_name, linewidth=2)
    plt.plot(idx, yhat_naive_inv[sample_idx], ":", label="Naive", linewidth=2)
    plt.title(f"Best vs Naive (sample {sample_idx})")
    plt.xlabel("Time"); plt.ylabel("PM2.5")
    plt.legend(); plt.tight_layout(); plt.show()

plot_best_vs_naive(0)
```

```
   Horizon (hr)        MAE       RMSE
0             1  18.452965  35.277242
1             2  24.606118  44.331347
2             3  29.404488  47.556350
3             4  34.174165  51.882753
4             5  38.650660  53.607737
5             6  40.256000  55.273378
6             7  42.801814  56.767517
7             8  43.082705  58.292801
8             9  44.647296  59.284180
9            10  44.406481  61.352809
Best model: GRU-128 (efficient alt.)
```









Model has been experiments:

1. LSTM-128 (capacity)

- Arch: LSTM(128) → Dense(24) with MAE loss.
- Why include: Strong single-layer baseline; 128 units give enough memory to learn diurnal + some multi-day structure.
- Strengths: Good amplitude vs smaller LSTMs; trains faster than stacked networks.
- Risks: Can overfit if validation is much worse than train; sometimes still conservative on peaks.

2. StackedLSTM 128→64 (deeper)

- Arch: LSTM(128, return_sequences=True) → LSTM(64) → Dense(24) with MAE.
- Why include: Deeper temporal hierarchy—first layer models short cycles (hourly), second layer captures longer regimes (multi-day/weekly).
- Strengths: Often lifts mid/long-horizon accuracy and peak timing.
- Risks: More parameters → longer training; needs early-stopping (and sometimes light dropout) to avoid overfit.

3. GRU-128 (efficient alt.)

- Arch: GRU(128) → Dense(24) with MAE.
- Why include: GRUs are simpler than LSTMs (fewer gates) and often generalise better with similar capacity.
- Strengths: Good accuracy with fewer parameters than comparable LSTM; in your runs it won (lowest MAE/RMSE).
- Risks: Like LSTM, may under-predict extreme spikes without extra context/features.

4. LSTM-128 + Dropout(0.1) (regularised)

- Arch: LSTM(128) → Dropout(0.1) → Dense(24) with MAE.
- Why include: Add light regularisation when train loss < val loss (signs of overfit).
- Strengths: Stabilises validation; good when stacked/single LSTM starts to memorize noise.
- Trade-off: Dropout can slightly reduce amplitude, so expect MAE to improve (or stay similar) but peaks to flatten a bit.

5. LSTM-128 (MSE for peaks)

- Arch: LSTM(128) → Dense(24) with MSE loss.
- Why include: Put more penalty on large errors to push the model toward higher peaks (may improve RMSE).
- Strengths: Sometimes closer to true maxima during episodes.
- Trade-off: Can hurt MAE and overall stability; in your results it under-performed the MAE models.

6. GRU-128 (no dropout)

- Arch: Same as #3, explicit note that there's no dropout.
- Why include: When train ≈ val, removing dropout can restore amplitude (less conservative forecasts).
- Strengths/risks: Slightly better peaks; watch validation for signs of overfit. Keep EarlyStopping.

#	Model	Loss	Test MAE	Test RMSE
1	**GRU-128 (no dropout)**	MAE	**45.33**	**64.49**
2	StackedLSTM 128→64 (deeper)	MAE	45.42	64.97
3	GRU-128 (efficient alt.)	MAE	45.61	64.74
4	LSTM-128 + Dropout(0.1) (reg.)	MAE	49.98	68.54
5	LSTM-128 (MSE for peaks)	**MSE**	54.39	70.82
6	LSTM-128 (capacity)	MAE	54.76	72.53
7	Naïve (repeat-last)	—	69.75	98.52

Best in this setting: GRU-128 (no dropout) — lowest MAE/RMSE and smooth horizon profile.

C) Horizon-wise behaviour (GRU-128, 21-day)

Errors increase with lead time (expected for direct multi-step): 1-hr: MAE ≈ 14.5, RMSE ≈ 33.3 → 24-hr: MAE ≈ 50–55, RMSE ≈ 70–76. Forecasts are most reliable in the first 6–12 hours.

D) What the results mean (explanation)

- GRU family handled diurnal/weekly structure with fewer parameters → solid generalisation.
- Deeper LSTM was competitive but marginally behind; dropout reduced overfit but also amplitude.
- MSE loss emphasised large errors but hurt overall accuracy, so MAE remained the best choice.
- Longer lookback captures stagnation/build-up events; beyond 14d the gains plateau on headline metrics.

E) Business impact

- Use case: next-day PM2.5 advisories, short-term traffic/emissions control, and clinic planning.
- Value: vs naïve, the chosen model cuts error by ~35–38%, especially improving the 0–12h window where actions matter most (morning school/commute, industrial shifts).
- Communications: publish horizon-wise KPIs and health thresholds (e.g., “Unhealthy” bands) to guide decisions.

F) Deployment considerations

- Data quality: monitor sensor uptime, gap rates; auto-apply ffill/bfill and flag outages.
- Drift & retraining: track rolling MAE/RMSE; retrain monthly or when drift exceeds a threshold.
- Calibration: optional validation-based scale correction to counter under-amplitude on extreme peaks.
- Robustness: clip negatives; cap implausible spikes; fall back to naïve when inputs are incomplete.
- Alerts: run hourly; keep inference <1s; log horizon-wise errors for continuous improvement.

One-line takeaway

Across multiple RNN variants and lookbacks, GRU-128 with a 21-day context delivers the best balance of accuracy and stability on Jan-2016, reducing error ~35–38% vs a naïve baseline and providing actionable 24-hour PM2.5 forecasts for public-health and operational decisions.

5. Role of GenAI

I have used GenAI to improve my code and to see if my code need any fix or improvement, the prompt has been used: "Please access to this notebook file and let me know what should I improve with my code".

ChatGPT was used to brainstorm ideas and list all of the steps must be done in each sections of the assignment.

Chat Gpt was used to troubleshoot error code and improve help to improve the accuracy of the model.

Chat GPT was used to write key ideas to brainstorm and list the work in a structured way.

While using GenAI: some obstacle has been meet with the over complicated while debugging and improve the code. I need to understand the code and remove part that does not used and teach in our lab session.